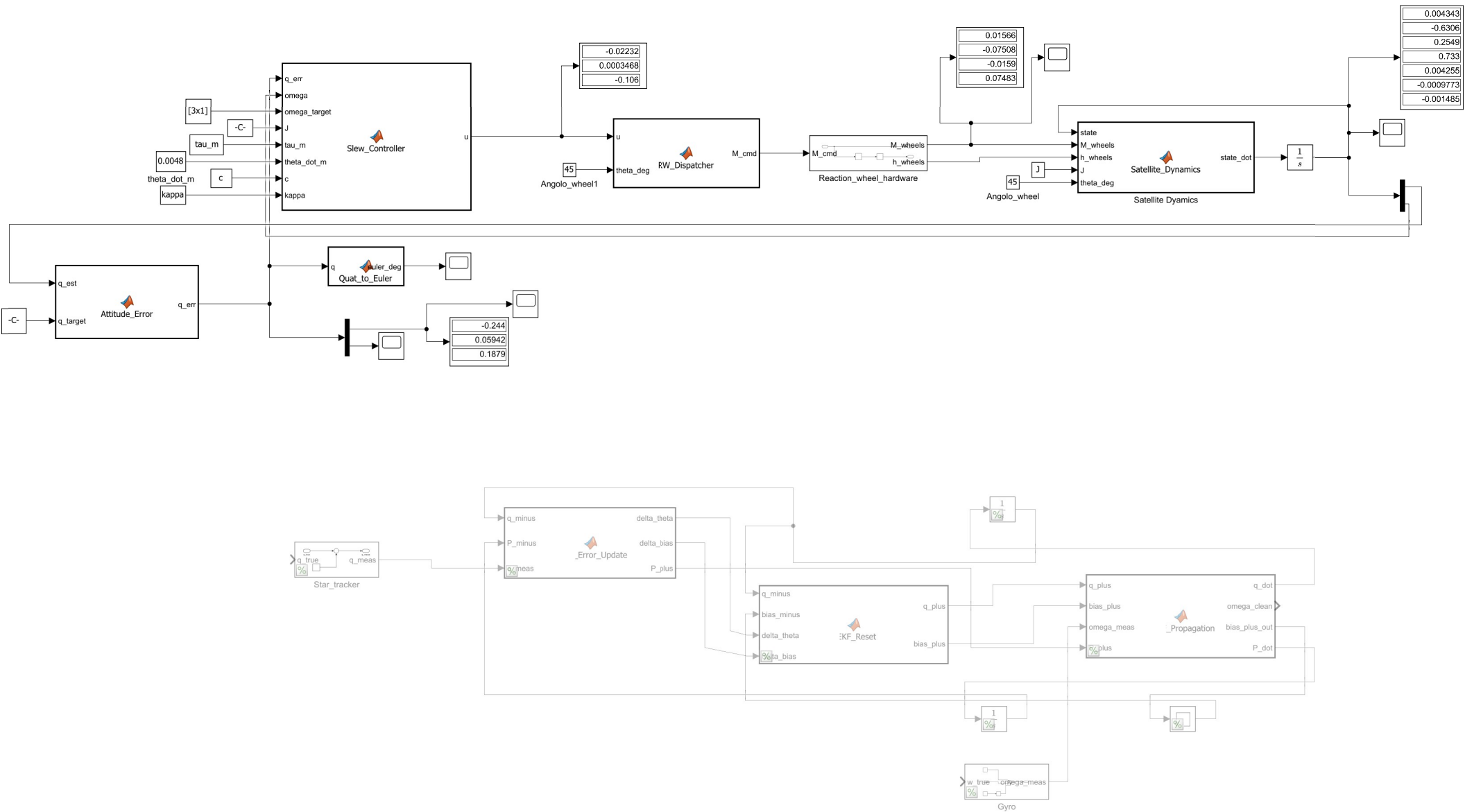
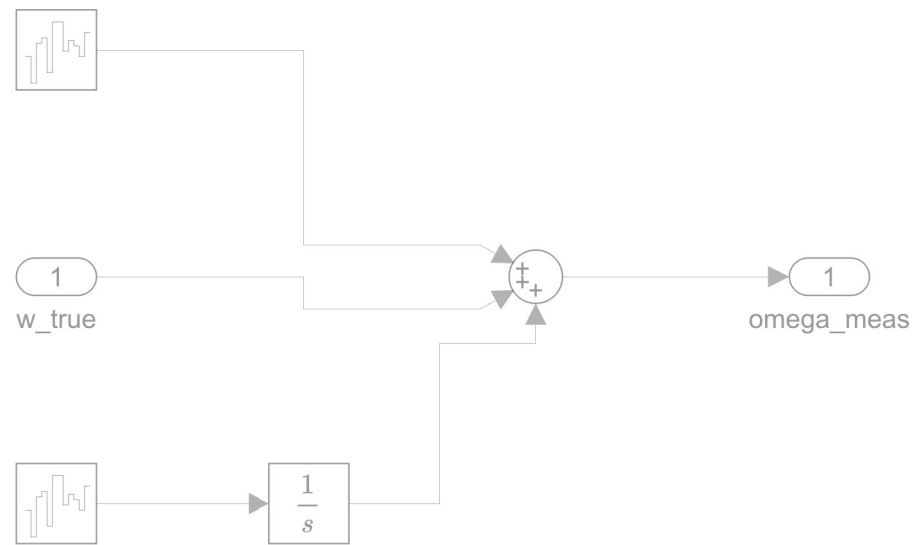






```

function [delta_theta, delta_bias, P_plus] = MEKF_Error_Update(q_minus, P_minus, q_meas)
% 1. MODELLO DI MISURA
H = [eye(3), zeros(3,3)];
sigma_ST = (10 / 3600) * (pi / 180); % Rumore Star Tracker (es. 10 arcosecondi)
R = (sigma_ST^2) * eye(3);

% 2. CALCOLO DELL'INNOVAZIONE (Errore di Quaternione)
q_inv = [-q_minus(1:3); q_minus(4)];
% Moltiplicazione q_meas (X) q_minus^(-1)
delta_q_meas = [q_inv(4)*q_meas(1:3) + q_meas(4)*q_inv(1:3) - cross(q_meas(1:3), q_inv(1:3));
                q_meas(4)*q_inv(4) - dot(q_meas(1:3), q_inv(1:3))];

if delta_q_meas(4) < 0
    delta_q_meas = -delta_q_meas; % Forziamo rotazione minima
end
y_k = 2 * delta_q_meas(1:3); % Vettore di errore (Rodrigues approssimato)

% 3. GUADAGNO DI KALMAN
S = H * P_minus * H' + R;
K = P_minus * H' / S;

% 4. AGGIORNAMENTO DELLO STATO DI ERRORE
delta_x_plus = K * y_k;
delta_theta = delta_x_plus(1:3);
delta_bias = delta_x_plus(4:6);

% 5. AGGIORNAMENTO COVARIANZA
P_plus = (eye(6) - K * H) * P_minus;
P_plus = 0.5 * (P_plus + P_plus'); % Forza simmetria matematica
end

```

```

function [q_plus, bias_plus] = MEKF_Reset(q_minus, bias_minus, delta_theta, delta_bias)
    % 1. AGGIORNAMENTO DEL BIAS
    bias_plus = bias_minus + delta_bias;

    % 2. AGGIORNAMENTO DEL QUATERNIONE (Moltiplicativo)
    q_err = [delta_theta / 2; 1];
    q_tmp = [q_minus(4)*q_err(1:3) + q_err(4)*q_minus(1:3) - cross(q_err(1:3), q_minus(1:3));
             q_err(4)*q_minus(4) - dot(q_err(1:3), q_minus(1:3))];

    % Normalizzazione Brute Force
    q_plus = q_tmp / norm(q_tmp);
end

```

```

function [q_dot, omega_clean, bias_plus_out, P_dot] = MEKF_Propagation(q_plus, bias_plus, omega_meas, P_plus)
    bias_plus_out = bias_plus; % Il bias si stima costante nella propagazione

    % 1. VELOCITÀ ANGOLARE DEPURATA DAL BIAS
    omega_clean = omega_meas - bias_plus;

    % 2. CINEMATICA DEL QUATERNIONE
    w = omega_clean;
    skew_w = [ 0, -w(3), w(2);
               w(3), 0, -w(1);
               -w(2), w(1), 0 ];

    q_v = q_plus(1:3); q_s = q_plus(4);
    q_v_dot = -0.5 * skew_w * q_v + 0.5 * q_s * w;
    q_s_dot = -0.5 * w' * q_v;
    q_dot = [q_v_dot; q_s_dot];

    % 3. PROPAGAZIONE DELLA COVARIANZA (Aggiunta Fondamentale)
    % Matrice della Dinamica F [6x6]
    F = [ -skew_w, -eye(3);
          zeros(3,3), zeros(3,3) ];

    % Matrice Rumore di Processo Q [6x6]
    sigma_v = sqrt(10) * 1e-6; % ARW dei giroscopi
    sigma_u = sqrt(10) * 1e-9; % RRW (instabilità del bias)
    Q = blkdiag((sigma_v^2)*eye(3), (sigma_u^2)*eye(3));

    % Derivata di Riccati
    P_dot = F * P_plus + P_plus * F' + Q;
end

```

```

function u = Slew_Controller(q_err, omega, omega_target, J, tau_m, theta_dot_m, c, kappa)
% 1. ERRORE DI ASSETTO E DI VELOCITÀ RELATIVA
q_e = q_err(1:3);
norm_qe = norm(q_e);

% La velocità relativa al bersaglio orbitale (Inseguimento)
delta_omega = omega - omega_target;

% Sicurezza: se l'errore è microscopico, spegni i motori e risparmia energia
if norm_qe < 1e-6
    u = [0; 0; 0];
    return;
end

% 2. CALCOLO DEI GUADAGNI DEL CONTROLLORE DI KJELLBERG
% C è il "freno" (Azione derivativa)
C = c * J;

% K è la "molla" proporzionale per ogni asse
k1 = c * (abs(q_e(1)) / norm_qe) * theta_dot_m;
k2 = c * (abs(q_e(2)) / norm_qe) * theta_dot_m;
k3 = c * (abs(q_e(3)) / norm_qe) * theta_dot_m;
K = diag([k1, k2, k3]) * J;

% 3. CALCOLO DELLA VELOCITÀ DESIDERATA (Senza divisioni per zero)
% Uso 'epsilon' per evitare che il calcolo esploda e per addolcire la frenata
epsilon = 1e-3;
coeff = (kappa * norm_qe) / (c * theta_dot_m);

v_in = zeros(3,1);
v_in(1) = coeff * (q_e(1) / (abs(q_e(1)) + epsilon));
v_in(2) = coeff * (q_e(2) / (abs(q_e(2)) + epsilon));
v_in(3) = coeff * (q_e(3) / (abs(q_e(3)) + epsilon));

% 4. SATURAZIONE INTERNA
% Taglia la richiesta matematica se l'errore iniziale di 140° è troppo grande
v_sat = max(min(v_in, 1), -1);

% 5. COPPIA TEORICA (Molla proporzionale + Freno derivativo)
tau = K * v_sat + C * delta_omega;

% 6. SATURAZIONE ESTERNA HARDWARE
% Se la coppia calcolata supera i limiti fisici delle ruote, la riduce
% in proporzione per non perdere la direzione della manovra di Slew
sigma = max(abs(tau) / tau_m);
if sigma > 1
    u = -tau / sigma;
else
    u = -tau;
end
end

```

```

function euler_deg = Quat_to_Euler(q)
% INGRESSO: q (Quaternione 4x1, convenzione Scalar-Last)
% USCITA: euler_deg (Vettore 3x1 con [Roll; Pitch; Yaw] in GRADI)

% 1. Estrazione degli elementi del quaternione
q1 = q(1);
q2 = q(2);
q3 = q(3);
q4 = q(4); % Elemento scalare

% 2. Calcolo dei soli elementi della Matrice di Rotazione (DCM) necessari
% Formule basate sulla trasformazione Quaternione -> DCM
M11 = q1^2 - q2^2 - q3^2 + q4^2;
M12 = 2 * (q1*q2 + q3*q4);
M13 = 2 * (q1*q3 - q2*q4);
M23 = 2 * (q2*q3 + q1*q4);
M33 = -q1^2 - q2^2 + q3^2 + q4^2;

% 3. Estrazione degli angoli di Eulero (Sequenza 3-2-1)

% ROLLIO (Roll / Phi) - Rotazione attorno asse X
roll = atan2(M23, M33);

% BECCHEGGIO (Pitch / Theta) - Rotazione attorno asse Y
% Aggiungiamo una protezione numerica per evitare che errori di
% arrotondamento portino l'argomento di asin fuori da [-1, 1],
% il che genererebbe numeri complessi e manderebbe in blocco Simulink.
sin_p = -M13;
sin_p = max(min(sin_p, 1), -1);
pitch = asin(sin_p);

% IMBARDATA (Yaw / Psi) - Rotazione attorno asse Z
yaw = atan2(M12, M11);

% 4. Conversione in GRADI e assemblaggio del vettore in uscita
euler_deg = [roll; pitch; yaw] * (180 / pi);
end

```

```

function q_err = Attitude_Error(q_est, q_target)
% 1. Calcola l'inverso del quaternion target (Convenzione Scalar-Last)
% L'inverso di un quaternion unitario coincide con il suo coniugato
q_target_inv = [-q_target(1:3); q_target(4)];

% 2. Moltiplicazione quaternionica: q_target_inv (X) q_est
% Applichiamo rigorosamente la formula del prodotto per la convenzione Scalar-Last
q_err_vec = q_est(4)*q_target_inv(1:3) + q_target_inv(4)*q_est(1:3) - cross(q_target_inv(1:3), q_est(1:3));
q_err_sca = q_target_inv(4)*q_est(4) - dot(q_target_inv(1:3), q_est(1:3));

q_err = [q_err_vec; q_err_sca];

% 3. Forziamo la rotazione minima (Shortest path)
% Evita che il satellite compia una rotazione maggiore di 180 gradi
if q_err(4) < 0
    q_err = -q_err;
end
end

```



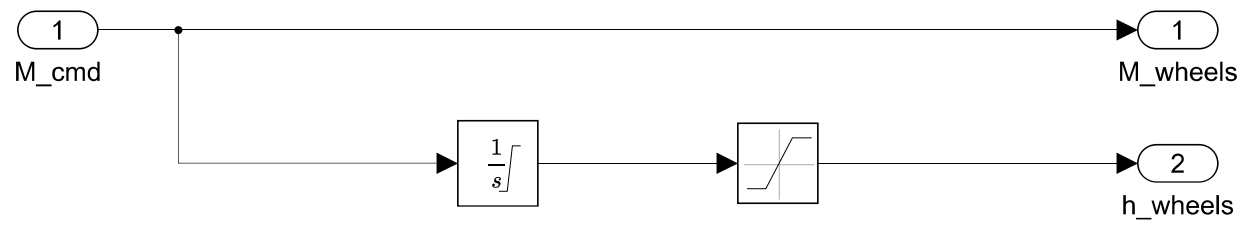
```

function M_cmd = RW_Dispatcher(u, theta_deg)
% u: Coppia desiderata sui 3 assi del satellite (dal Slew Controller)
% theta: angolo di inclinazione delle ruote (es. 45 * pi/180)
theta = theta_deg * (pi/180);
% Matrice di Configurazione hardware a 4 ruote inclinate
Aw = [cos(theta),      0, -cos(theta),      0;
      sin(theta), sin(theta), sin(theta), sin(theta);
      0, -cos(theta),      0, cos(theta)];

% Calcolo della Right Pseudoinverse
Aw_pinv = pinv(Aw);

% Distribuzione della coppia: minimizza l'energia spesa dalle 4 ruote
M_cmd = Aw_pinv * (-u);
end

```



```
function state_dot = Satellite_Dynamics(state, M_wheels, h_wheels, J, theta_deg)
```

```
%% 0. MESSA IN SICUREZZA DELLE DIMENSIONI (Evita gli errori di compilazione Simulink)
% Dichiariamo subito la dimensione dell'uscita
state_dot = zeros(7,1);
```

```
% Forziamo tutti i segnali in ingresso a essere vettori colonna (Nx1)
state = state(:);
M_wheels = M_wheels(:);
h_wheels = h_wheels(:);
```

```
% Assicuriamoci che l'angolo sia uno scalare
theta_deg = theta_deg(1);
```

```
%% 1. ESTRAZIONE DELLO STATO
% Quaternione (Convenzione Scalar-Last)
q = state(1:4);
% Velocità angolare [rad/s]
w = state(5:7);
```

```
%% 2. MATRICE DELLA GEOMETRIA DELLE RUOTE
theta = theta_deg * (pi/180);
```

```
% Matrice di attuazione Aw per la configurazione a 4 ruote inclinate
Aw = [ cos(theta),      0,      -cos(theta),      0;
       sin(theta),  sin(theta),  sin(theta),  sin(theta);
       0,      -cos(theta),      0,      cos(theta)];
```

```
% Proiezione dai 4 assi delle ruote ai 3 assi del corpo
M_contr = Aw * M_wheels;
h_w_contr = Aw * h_wheels;
```

```
%% 3. DINAMICA ROTAZIONALE (Equazioni di Eulero Accoppiate)
skew_w = [ 0,      -w(3),      w(2);
           w(3),      0,      -w(1);
          -w(2),      w(1),      0 ];
% bisogna inserire il contributo di gravity gradient e
% external torque nella formula sotto
```

```
w_dot = J \ (-M_contr - skew_w * (J * w + h_w_contr));
```

```
%% 4. CINEMATICA D'ASSETTO (Derivata del Quaternione)
q_v = q(1:3);
q_s = q(4);
```

```
q_v_dot = -0.5 * skew_w * q_v + 0.5 * q_s * w;
q_s_dot = -0.5 * w' * q_v;
```

```
q_dot = [q_v_dot; q_s_dot];
```

```
%% 5. ASSEMBLAGGIO DELL'USCITA
state_dot = [q_dot; w_dot];
% Calcolo dell'errore di norma del quaternione
norm_err = 1 - (q(1)^2 + q(2)^2 + q(3)^2 + q(4)^2);
k_norm = 0.05; % Abbassato da 10 per garantire stabilità numerica
```

```
% Aggiungiamo il termine di Baumgarte alla derivata
q_dot = [q_v_dot; q_s_dot] + k_norm * norm_err * q;
```

```
% Vettore di stato derivato in uscita
state_dot = [q_dot; w_dot];
```